

ADVANCED DATABASE MANAGEMENT SYSTEM

Database Syntax Quick-Reference | UCSPC0632 | T.Y. CSE - VI | 2025-26

MongoDB - Document Database (NoSQL)

Experiment: Experiments 7 & 8 - CRUD on Document Database | Course Outcome: CO3

WHERE TO RUN

Open: MongoDB Compass (GUI) OR Terminal / Command Prompt

Step 1: Install MongoDB Community Server from [mongodb.com](https://www.mongodb.com)

Step 2: Start service -> `sudo systemctl start mongod` (Linux)

Step 2: Start service -> `net start MongoDB` (Windows CMD as Admin)

Step 3: Open MongoDB Shell -> type: `mongosh` in Terminal / CMD

Step 4: All commands below run inside the `mongosh` prompt

GUI Alt: Open MongoDB Compass -> Connect -> Open Mongosh tab at bottom

Database & Collection

Syntax / Command	One-Line Description
<code>show dbs</code>	List all databases
<code>use myDB</code>	Switch to / create database 'myDB'
<code>db</code>	Show current database name
<code>show collections</code>	List collections in current DB
<code>db.createCollection('students')</code>	Explicitly create a collection
<code>db.students.drop()</code>	Delete a collection
<code>db.dropDatabase()</code>	Drop the current database

INSERT (Create)

Syntax / Command	One-Line Description
<code>db.students.insertOne({name: 'Alice', age: 21})</code>	Insert a single document
<code>db.students.insertMany([{name: 'Bob'}, {name: 'Carol'}])</code>	Insert multiple documents

SELECT (Read)

Syntax / Command	One-Line Description
<code>db.students.find()</code>	Retrieve all documents
<code>db.students.find().pretty()</code>	Retrieve all - formatted output
<code>db.students.find({age: 21})</code>	Filter where age = 21
<code>db.students.find({age: {\$gt: 20}})</code>	Filter age > 20
<code>db.students.find({}, {name: 1, _id: 0})</code>	Project only 'name' field
<code>db.students.findOne({name: 'Alice'})</code>	Return first matching document
<code>db.students.find().sort({age: -1})</code>	Sort descending by age
<code>db.students.find().limit(5)</code>	Return first 5 documents
<code>db.students.countDocuments({age: {\$gt: 20}})</code>	Count matching documents

UPDATE

Syntax / Command	One-Line Description
<code>db.students.updateOne({name: 'Alice'}, {\$set: {age: 22}})</code>	Update one field in first match
<code>db.students.updateMany({age: {\$lt: 20}}, {\$set: {status: 'minor'}})</code>	Update all matching docs

<code>db.students.replaceOne({name: 'Bob'}, {name: 'Bob', age: 25})</code>	Replace entire document
--	-------------------------

DELETE	
Syntax / Command	One-Line Description
<code>db.students.deleteOne({name: 'Alice'})</code>	Delete first matching document
<code>db.students.deleteMany({age: {\$lt: 18}})</code>	Delete all matching documents
<code>db.students.deleteMany({})</code>	Delete ALL documents in collection

Aggregation	
Syntax / Command	One-Line Description
<code>db.students.aggregate([{\$group: {_id: '\$dept', total: {\$sum: 1}}]})</code>	Group & count by dept
<code>db.students.aggregate([{\$match: {age: {\$gte: 21}}]})</code>	Filter stage in pipeline
<code>db.students.aggregate([{\$sort: {age: 1}}]})</code>	Sort stage in pipeline

Oracle DB - Relational / Object-Relational Database

Experiment: Experiments 1,2,3 & 4 - Complex Types, Inheritance, Partitioning | Course Outcome: CO1, CO2

WHERE TO RUN

Open: Oracle SQL Developer (GUI) OR SQL*Plus (Terminal)
 Step 1: Install Oracle Database 21c/19c XE from [oracle.com](https://www.oracle.com/in/database/technologies/quickstart-downloads.html)
 Step 2: Install Oracle SQL Developer from [oracle.com/sqldeveloper](https://www.oracle.com/in/developer/)
 Step 3 (GUI): Open SQL Developer -> New Connection -> Enter user/pass -> Connect
 Step 3 (CLI): Open Terminal/CMD -> type: `sqlplus username/password`
 Step 4: All SQL commands run in the SQL Developer Worksheet or SQL*Plus prompt
 Default user: SYSTEM | Default SID: XE | Port: 1521

Connection & Schema	
Syntax / Command	One-Line Description
<code>sqlplus user/pass@//host:1521/SID</code>	Connect via SQL*Plus CLI
<code>SHOW USER;</code>	Display current connected user
<code>SELECT table_name FROM user_tables;</code>	List all tables of current user
<code>DESC table_name;</code>	Describe table structure
<code>SELECT * FROM v\$version;</code>	Show Oracle version info

Complex / Object Types (CO2)	
Syntax / Command	One-Line Description
<code>CREATE TYPE addr_t AS OBJECT(street VARCHAR2(50), city VARCHAR2(30));</code>	Define an object type
<code>CREATE TYPE phone_list_t AS VARRAY(5) OF VARCHAR2(15);</code>	VARRAY - fixed-max collection
<code>CREATE TYPE course_tab_t AS TABLE OF VARCHAR2(30);</code>	Nested Table type
<code>CREATE TABLE students(sid NUMBER, addr addr_t, phones phone_list_t) NESTED TABLE phones STORE AS phones_nt;</code>	Table with nested type
<code>INSERT INTO students VALUES(1,addr_t('MG Rd', 'Kolhapur'),phone_list_t('9876543210'));</code>	Insert with object constructor
<code>SELECT s.addr.city FROM students s WHERE s.sid=1;</code>	Access object attribute via dot notation

Inheritance / OODBMS (CO2)	
Syntax / Command	One-Line Description
<code>CREATE TYPE person_t AS OBJECT(pid NUMBER,name VARCHAR2(40)) NOT FINAL;</code>	Base type (NOT FINAL enables subtyping)
<code>CREATE TYPE student_t UNDER person_t(rollno NUMBER,dept VARCHAR2(20));</code>	Sub-type inheriting from person_t
<code>CREATE TYPE employee_t UNDER person_t(empid NUMBER,salary NUMBER);</code>	Another sub-type of person_t
<code>CREATE TABLE persons OF person_t;</code>	Object table for base type
<code>INSERT INTO persons VALUES(student_t(1, 'Alice', 101, 'CSE'));</code>	Insert sub-type row into base table
<code>SELECT VALUE(p) FROM persons p WHERE VALUE(p) IS OF TYPE(student_t);</code>	Query specific sub-type
<code>SELECT TREAT(VALUE(p) AS student_t).dept FROM persons p;</code>	Cast and access sub-type attribute

Range Partitioning (CO1)	
Syntax / Command	One-Line Description
<code>CREATE TABLE sales(id NUMBER,sdate DATE,amt NUMBER) PARTITION BY RANGE(sdate)(PARTITION p2023 VALUES LESS THAN(TO_DATE('2024-01-01','YYYY-MM-DD')), PARTITION p2024 VALUES LESS THAN(MAXVALUE));</code>	Range partition on date column
<code>SELECT partition_name FROM user_tab_partitions WHERE table_name='SALES';</code>	List partitions of a table
<code>SELECT * FROM sales PARTITION(p2023);</code>	Query a specific named partition
<code>ALTER TABLE sales ADD PARTITION p2025 VALUES LESS THAN(TO_DATE('2026-01-01','YYYY-MM-DD'));</code>	Add a new range partition

Hash Partitioning (CO1)	
Syntax / Command	One-Line Description
<code>CREATE TABLE employees(eid NUMBER,ename VARCHAR2(40),dept VARCHAR2(20)) PARTITION BY HASH(eid) PARTITIONS 4;</code>	Hash partition into 4 buckets
<code>SELECT * FROM employees PARTITION(SYS_P1);</code>	Query a hash partition by system name
<code>ALTER TABLE employees ADD PARTITION;</code>	Add an extra hash partition

DML / DDL Basics	
Syntax / Command	One-Line Description
<code>CREATE TABLE t(id NUMBER PRIMARY KEY, name VARCHAR2(40));</code>	Create a basic table
<code>INSERT INTO t VALUES(1, 'Alice');</code>	Insert a row
<code>SELECT * FROM t WHERE id=1;</code>	Select with filter
<code>UPDATE t SET name='Bob' WHERE id=1;</code>	Update a field
<code>DELETE FROM t WHERE id=1;</code>	Delete a row
<code>COMMIT;</code>	Persist transaction to disk
<code>ROLLBACK;</code>	Undo uncommitted changes

WHERE TO RUN

Option A - Local Install: Open Terminal (Linux/Mac) or CMD (Windows)

Step 1: Install Java JDK 11+ then install Apache Cassandra

Step 2: Start Cassandra -> sudo systemctl start cassandra (Linux)

Step 3: Open CQL Shell -> type: cqlsh in terminal

Option B - Astra DB (Cloud): Go to astra.datastax.com -> Create DB

Step 1: Create free Astra account at astra.datastax.com

Step 2: Create Serverless (Non-Vector) Database -> choose region

Step 3: Open CQL Console tab inside Astra dashboard

Step 4: All CQL commands below run in the cqlsh> or Astra CQL Console

Keyspace (= Database)

Syntax / Command	One-Line Description
DESCRIBE keyspaces;	List all keyspaces
CREATE KEYSPACE college WITH replication={'class':'SimpleStrategy','replication_factor':1};	Create keyspace with replication
USE college;	Switch to a keyspace
DROP KEYSPACE college;	Delete a keyspace

Table Operations

Syntax / Command	One-Line Description
DESCRIBE TABLES;	List tables in current keyspace
CREATE TABLE students(sid UUID PRIMARY KEY,name TEXT,age INT,dept TEXT);	Create table with UUID primary key
DESCRIBE students;	Show table structure / schema
ALTER TABLE students ADD email TEXT;	Add a new column to table
DROP TABLE students;	Delete a table

INSERT (Create)

Syntax / Command	One-Line Description
INSERT INTO students(sid,name,age,dept) VALUES(uuid(),'Alice',21,'CSE');	Insert row with auto-generated UUID
INSERT INTO students(sid,name,age) VALUES(now(),'Bob',22) USING TTL 86400;	Insert with TTL - expires after 1 day

SELECT (Read)

Syntax / Command	One-Line Description
SELECT * FROM students;	Retrieve all rows
SELECT name,dept FROM students WHERE sid=;	Select columns by primary key
SELECT * FROM students LIMIT 5;	Limit result to 5 rows
SELECT COUNT(*) FROM students;	Count all rows in table
SELECT * FROM students WHERE dept='CSE' ALLOW FILTERING;	Filter non-PK column (needs ALLOW FILTERING)

UPDATE

Syntax / Command	One-Line Description
UPDATE students SET age=22 WHERE sid=;	Update a column value by primary key
UPDATE students USING TTL 3600 SET dept='IT' WHERE sid=;	Update with TTL expiry

DELETE

Syntax / Command	One-Line Description
DELETE FROM students WHERE sid=;	Delete row by primary key

<code>DELETE email FROM students WHERE sid=;</code>	Delete only a specific column value
---	-------------------------------------

Indexes	
Syntax / Command	One-Line Description
<code>CREATE INDEX ON students(dept);</code>	Secondary index for non-PK filtering
<code>DROP INDEX students_dept_idx;</code>	Remove a secondary index
<code>SELECT TOKEN(sid),name FROM students;</code>	View partition token values

Neo4j - Graph Database (NoSQL)

Experiment: Experiments 10 & 11 - CRUD on Graph Database (Cypher Query Language) | Course Outcome: CO3

WHERE TO RUN

Open: Neo4j Desktop (GUI) OR Neo4j Browser (Web UI)

Step 1: Download Neo4j Desktop from neo4j.com/download

Step 2: Open Neo4j Desktop -> Add -> Local DBMS -> Set password -> Start

Step 3: Click Open button -> Neo4j Browser opens in Chrome/Firefox

Step 4: In the top bar of Neo4j Browser type Cypher commands and press Ctrl+Enter

Alt: Neo4j AuraDB Free Cloud -> aura.neo4j.io -> Create Free Instance -> Open Browser

All Cypher commands below run in the Neo4j Browser query bar

Node Operations	
Syntax / Command	One-Line Description
<code>CREATE (n:Student {name:'Alice',age:21})</code>	Create a node with label and properties
<code>MATCH (n:Student) RETURN n</code>	Retrieve all Student nodes
<code>MATCH (n:Student {name:'Alice'}) RETURN n</code>	Find node by specific property
<code>MATCH (n:Student) RETURN n.name, n.age</code>	Return specific property values
<code>MATCH (n:Student) RETURN count(n)</code>	Count all Student nodes

Update & Delete Nodes	
Syntax / Command	One-Line Description
<code>MATCH (n:Student {name:'Alice'}) SET n.age=22 RETURN n</code>	Update a node property
<code>MATCH (n:Student {name:'Alice'}) SET n.grade='A' RETURN n</code>	Add new property to node
<code>MATCH (n:Student {name:'Alice'}) REMOVE n.grade RETURN n</code>	Remove a property from node
<code>MATCH (n:Student {name:'Alice'}) DELETE n</code>	Delete node (must have no relations)
<code>MATCH (n:Student {name:'Alice'}) DETACH DELETE n</code>	Delete node AND all its relationships
<code>MATCH (n:Student) DETACH DELETE n</code>	Delete ALL Student nodes and relations

Relationship Operations	
Syntax / Command	One-Line Description
<code>MATCH (a:Student {name:'Alice'}),(b:Course {title:'DBMS'}) CREATE (a)-[:ENROLLED_IN]->(b)</code>	Create a directed relationship
<code>MATCH (a:Student {name:'Alice'}),(b:Student {name:'Bob'}) CREATE (a)-[:FRIENDS_WITH {since:2023}]->(b)</code>	Relationship with property
<code>MATCH (s:Student)-[:ENROLLED_IN]->(c:Course) RETURN s.name,c.title</code>	Traverse and return relationship data

<code>MATCH (s:Student)-[r:ENROLLED_IN]->() DELETE r</code>	Delete all ENROLLED_IN relationships
<code>MATCH ()-[r:FRIENDS_WITH {since:2023}]-() RETURN r</code>	Find relationships by property

Advanced Queries	
Syntax / Command	One-Line Description
<code>MATCH (s:Student)-[:ENROLLED_IN]->(c:Course) WHERE c.title='DBMS' RETURN s.name</code>	Filter via relationship traversal
<code>MATCH p=(s:Student)-[*1..3]-(n) RETURN p</code>	Variable-length path 1 to 3 hops
<code>MATCH (s:Student) RETURN s.name ORDER BY s.age DESC LIMIT 5</code>	Sort descending and limit results
<code>MATCH (s:Student) WHERE s.age > 20 RETURN s.name, s.age</code>	WHERE clause filter on property

Indexes & Constraints	
Syntax / Command	One-Line Description
<code>CREATE INDEX FOR (s:Student) ON (s.name)</code>	Create index on a node property
<code>CREATE CONSTRAINT FOR (s:Student) REQUIRE s.rollno IS UNIQUE</code>	Unique constraint on property
<code>SHOW INDEXES</code>	List all existing indexes
<code>DROP INDEX index_name</code>	Drop an index by name

Utility	
Syntax / Command	One-Line Description
<code>MERGE (n:Student {rollno:101}) ON CREATE SET n.name='Dave'</code>	Create only if node does not exist
<code>CALL db.labels()</code>	List all node labels in database
<code>CALL db.relationshipTypes()</code>	List all relationship types
<code>EXPLAIN MATCH (n:Student) RETURN n</code>	Show query execution plan

Riak - Key-Value Database (NoSQL)

Experiment: Experiments 5 & 6 - Installation & CRUD on Key-Value Database | Course Outcome: CO3

WHERE TO RUN

Riak runs **ONLY** on Linux. Use one of the options below:
Option A - Native Ubuntu/Linux: Open Terminal directly
Option B - Windows with WSL2 (Ubuntu on Windows):
Step 1: Open PowerShell as Admin -> run: `wsl --install`
Step 2: Restart PC -> Ubuntu will auto-install from Microsoft Store
Step 3: Set Linux username & password when prompted
Step 4: Open Ubuntu app from Start Menu (this is your Linux terminal)
Step 5: Install Riak inside this Ubuntu terminal (see Installation section)
Option C - VirtualBox: Install Ubuntu 20.04 VM -> open VM Terminal
All riak / curl commands below run inside the Ubuntu/Linux Terminal

WINDOWS USERS - Integrating Ubuntu via WSL2 (Windows Subsystem for Linux)

PowerShell (Admin) -> `wsl --install`

PowerShell (Admin) -> `wsl --install -d Ubuntu-20.04`

After restart, Ubuntu app opens -> create username & password

Open Ubuntu from Start Menu anytime to get Linux terminal

Check Ubuntu version inside terminal -> `lsb_release -a`

Update packages -> <code>sudo apt update && sudo apt upgrade -y</code>
Enable WSL2 (if not default) -> <code>wsl --set-default-version 2</code>
Check WSL version -> <code>wsl -l -v</code>
Riak install & all curl commands run inside this Ubuntu terminal

STEP 1 - Install Riak on Ubuntu / WSL Ubuntu	
Syntax / Command	One-Line Description
<code>sudo apt-get update</code>	Update Ubuntu package list
<code>sudo apt-get install -y curl apt-transport-https gnupg2</code>	Install required tools
<code>curl -s https://packagecloud.io/install/repositories/basho/riak/script.deb.sh sudo bash</code>	Add Riak official repo to apt
<code>sudo apt-get install riak</code>	Install Riak KV package
<code>riak version</code>	Verify Riak is installed correctly

STEP 2 - Start & Verify Riak	
Syntax / Command	One-Line Description
<code>sudo riak start</code>	Start the Riak server
<code>sudo riak ping</code>	Check if Riak is running -> should reply: pong
<code>sudo riak status</code>	View detailed Riak node status
<code>sudo riak-admin status</code>	Full cluster and ring health info
<code>sudo riak-admin member-status</code>	Show ring membership of nodes
<code>sudo riak stop</code>	Stop the Riak server
<code>sudo riak restart</code>	Restart the Riak server

STEP 3 - Enable Search (optional)	
Syntax / Command	One-Line Description
<code>sudo riak-admin bucket-type create default</code>	Create default bucket type
<code>sudo riak-admin bucket-type activate default</code>	Activate default bucket type
<code>sudo riak-admin transfers</code>	Show active data transfer status

CRUD via HTTP API (curl in Ubuntu Terminal)	
Syntax / Command	One-Line Description
<code>curl http://localhost:8098/ping</code>	Test if Riak HTTP API is responding
<code>curl http://localhost:8098/buckets?buckets=true</code>	List all buckets in Riak
<code>curl http://localhost:8098/buckets/students/keys?keys=true</code>	List all keys in 'students' bucket
<code>curl -X PUT http://localhost:8098/buckets/students/keys/alice -H 'Content-Type:application/json' -d '{"age":21,"dept":"CSE"}'</code>	Store key 'alice' with JSON value
<code>curl -X GET http://localhost:8098/buckets/students/keys/alice</code>	Retrieve value of key 'alice'
<code>curl -X PUT http://localhost:8098/buckets/students/keys/alice -H 'Content-Type:application/json' -d '{"age":22,"dept":"CSE"}'</code>	Update key 'alice' with new value
<code>curl -X DELETE http://localhost:8098/buckets/students/keys/alice</code>	Delete key 'alice' from bucket

<pre>curl -X POST http://localhost:8098/buckets/students/keys -H 'Content-Type:application/json' -d '{"name":"Bob"}'</pre>	Store with auto-generated key
--	-------------------------------

Bucket Properties (curl)	
Syntax / Command	One-Line Description
<pre>curl http://localhost:8098/buckets/students/props</pre>	Get properties of 'students' bucket
<pre>curl -X PUT http://localhost:8098/buckets/students/props -H 'Content-Type:application/json' -d '{"props":{"n_val":3}}'</pre>	Set replication factor n_val to 3
<pre>curl http://localhost:8098/stats</pre>	View Riak server statistics

Python Client (inside Ubuntu Terminal)	
Syntax / Command	One-Line Description
<pre>sudo apt-get install python3-pip</pre>	Install pip in Ubuntu / WSL
<pre>pip3 install riak</pre>	Install Riak Python client library
<pre>import riak</pre>	Import Riak in Python script
<pre>client = riak.RiakClient(host='127.0.0.1', pb_port=8087)</pre>	Connect to Riak via Protocol Buffers
<pre>bucket = client.bucket('students')</pre>	Get or create a bucket object
<pre>obj = bucket.new('alice', data={'age':21}); obj.store()</pre>	Create and store a new object
<pre>obj = bucket.get('alice'); print(obj.data)</pre>	Retrieve object by key
<pre>obj.data['age'] = 22; obj.store()</pre>	Update object data and save
<pre>bucket.delete('alice')</pre>	Delete object by key from bucket

Secondary Indexes - 2i (curl)	
Syntax / Command	One-Line Description
<pre>curl -X PUT ../buckets/students/keys/alice -H 'x-riak-index-dept_bin: CSE' -d '{}'</pre>	Store object with binary secondary index
<pre>curl http://localhost:8098/buckets/students/index/dept_bin/CSE</pre>	Query all keys where dept = CSE
<pre>curl http://localhost:8098/buckets/students/index/age_int/18/25</pre>	Range query: age between 18 and 25